

A Formal Model for REST

Prof. Dr. Peter Braun

6. Januar 2025

1 Titlepage

A Formal Model for REST

Prof. Dr. Peter Braun



No Text – maybe some music :-)

2 Learning Goals

Learning Goals

- Mathematical model of a Finite State Machine
- How the World Wide Web works
- Theoretical model of the World Wide Web
- Practical example of a RESTful communication

Welcome to this unit, in which we will explore the theory of the REST paradigm. In previous units, we learned the foundational concepts of the REST architecture. You have discovered that in a distributed system built according to REST principles, the emphasis is on the data exchanged between the client and server. You have also learned that only four basic operations can be performed on any type of data. You are familiar with the acronym CRUD, which stands for creating, reading, updating, and deleting data. Additionally, you have learned about the hypermedia aspect of these systems: the server informs the client in each response message through hyperlinks about what further or subsequent operations are possible. In computer science, we know various theoretical models for automats, including the Turing machine, pushdown automaton, and finite state machine. You might not have studied all of them, but you are likely familiar with the simplest one: Finite State Machines. You should recall this model from lectures on Computer Architecture, where it was referred to as Moore or Mealy Machines, or from Software Engineering and UML state diagrams. These models are employed to provide a detailed description of a system's dynamic behavior. While some of these models emerged from areas now more commonly known as theoretical computer science, all of them have practical applications. Today, we want to explore which model best describes how the World Wide Web operates. We are doing this to revisit the hypermedia property as one of the foundational principles of REST. I aim to enhance your understanding of this principle of REST and your motivation to apply it in your projects. We will start by examining the formal definition of finite-state machines. Next, I will clarify how the World Wide Web functions, and I will argue that finite-state machines provide a suitable model for the World Wide Web. Finally, at the end of the unit, we will look at a practical example.

3 Definition of Finite-state Machines

Definition of Finite State Machines

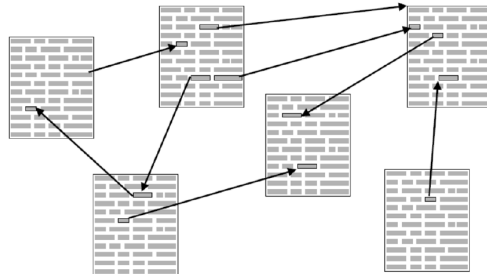
A finite State Machine (FSM) consists of:

- Finite, non-empty set of states $S = q_0, q_1, \dots, q_n$
- Finite, non-empty set of input symbols Σ
- One initial state $S_0 \in S$
- Finite set of final states $F \subseteq S$
- Transition relation $\delta \subseteq S \times \Sigma \times S$

Let's start with the theoretical framework of finite state machines. An automaton of this type comprises a finite set of states, denoted as S . The automaton processes input consisting of symbols, collectively known as Σ . Initially, the machine begins in a defined starting state and ceases operation once the input has been completely processed. The input is accepted if the machine reaches a final state in F . This definition arises from the field of acceptors. In this context, such a machine determines whether an input is valid within a formal language framework. The processing occurs as follows: the machine is always in a specific state. The next symbol from the input is read, and based on the transition relation Δ , a subsequent state is determined for this combination of state and symbol. This concludes one processing step, and the next symbol is then processed. This continues until the entire input has been processed, which may also be infinitely long. This model is straightforward yet significantly important in practice. It is represented as state transition diagrams within UML, and many dynamic processes in software and hardware systems can be described as finite state machines. Finite state machines play a crucial role in compiler construction, as they enable the determination of whether a word belongs to a regular language. Regular languages are essential for describing tokens, such as permissible variable names.

4 World Wide Web

World Wide Web



- Client-Server System
- Non-linear documents with hyperlinks

Let's recall what you have already learned about the World Wide Web. The World Wide Web is, as we understand it, the most extensive distributed system currently in the world. It operates strictly on the principle of client and server, with web browsers serving as the clients and the so-called web servers functioning as the servers. Documents on the World Wide Web are interconnected through hyperlinks, with documents containing these links referred to as hypertext. There are various uses for such hyperlinks. Additionally, hyperlinks can enable the reader to explore the content more deeply. By incorporating hyperlinks, the linear reading of documents is enhanced with the option to jump to other sections or even to entirely different documents for further information. This gives the reader new freedom to choose what they want to read. However, it's important to recognize that this freedom is ultimately constrained by what is available in the documents. The reader can only access linked information if the author of a document has provided a hyperlink. The document's author offers the reader options for obtaining further details while leaving the choice to accept that offer up to them. Unfortunately, this aspect of hypertext and hyperlinks is often not consistently pursued on the web pages currently published. Wikipedia serves as a positive exception in this regard.

5 Intermediate Result

Intermediate Result

- Web Pages are like states
- Hyperlinks are like transitions

Let us now attempt a first connection between finite state machines and the World Wide Web. The central focus of the finite-state machine model is the states. The machine is in exactly one state at any given time. What might this state represent in the World Wide Web? First, we must restrict ourselves to looking at a client or end-user. The user is viewing exactly one document in their browser, which is irrelevant to where it originated. Thus, the user is in the state: "Looking at document X." To transition from one state, i.e., "Looking at document X," to another state, i.e., "Looking at document Y," the user must use a hyperlink. Therefore, hyperlinks are related to the state transition function δ . I have already provided you with an initial clue.

6 Finite-state Machine for Web Sites

Finite State Machine for Web Sites

We can model a Web Site using a Finite State Machine:

- $S = \{S_{Wikipedia}, S_{Hypertext}, S_{WWW}, \dots\}$
- $\Sigma = \{E_{URL_1}, E_{URL_2}, E_{URL_3}, \dots\}$
- $S_0 = S_{Wikipedia}$
- $F = \emptyset$
- $\delta = \{(S_{Hypertext}, E_{URL_1}, S_{WWW}), \dots\}$

Here, event E_{URL} refers to the user clicking on the hyperlink that leads to the given URL.

We can view the documents as states and set Sigma as “clicking” on the hyperlinks. The clicks on the links are the events that move the automaton’s state forward. State transitions result from the possible hyperlinks present in a document. By the way, the start state S_0 depends on the browser’s configuration and is, of course, different for every user. However, there is a start state; in any case, a document appears at the beginning after starting the browser, such as a search engine’s homepage. Strictly speaking, regarding the amount of sigma, we must also allow the user to enter an address in their browser—unfortunately, not everything can be accessed through hyperlinks.

7 REST Application as Finite State Machine

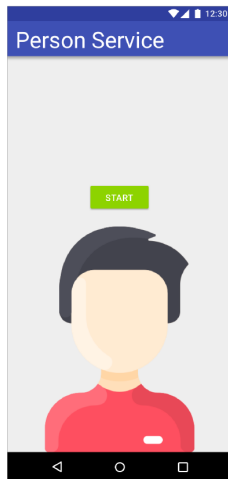
REST Application as Finite State Machine

- Clients have a state
- Clients transfer the state to the server
- Server replies with subsets of δ

Suppose the World Wide Web can be described as a finite automaton from the perspective of a client or end-user. In that case, any REST-compliant system can also be modeled as a finite automaton because REST is the architecture of the World Wide Web. We will now examine this in detail. In doing so, we will establish a connection for the first time between the user interface for the end-user on the client and the service on the server. REST is an architectural paradigm for distributed systems, always referring to client and server. The goal is for you to understand the following points in detail through the subsequent explanations: The client exists in a state in the sense of a finite state machine. The server is stateless; it knows nothing about the state of the clients currently accessing it. Upon access, the client transmits all relevant information about its state to the server. It sends an HTTP request message with a specific HTTP verb and a URI to achieve this. It may also send additional data and metadata. The server processes this request message; again, it knows nothing about the client's state apart from what is in the request message and sends a response back to the client. In the response message, the server informs the client about potential follow-up actions, which essentially comprise an excerpt from the relation delta concerning the client's current state. The possible follow-up actions are hyperlinks similar to HTML pages, yet they still include relation names for better understanding by the client. The client can associate the relation names with concrete actions, such as pressing a button, which it can then present to the end user.

8 Application Start

Application Start



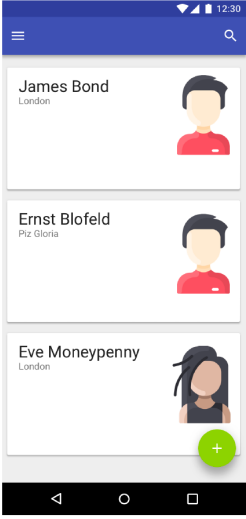
```
GET /demo/api HTTP/1.1
Host: http://localhost:8080
Accept: */*
```

```
HTTP/1.1 200 OK
Date: Mon, 17 Aug 2020 12:30:01 GMT
Link: <.../demo/api/persons>; rel="getAllPersons"
```

Now, let's look at how to deal with a REST API using a concrete example. We will consistently argue from the perspective of a client application. When the application starts, it is in the startup state and displays a splash screen. The first request to the server is made to the only URL the client knows and has programmed. On the server, this initial address is referred to as the dispatcher. In the response message, the server sends the client information about the next action that the client should take. Here, only a hyperlink is provided with the relation name "getAllPersons". If there is only one subsequent state, the client can either take this path immediately or wait for an action from the end user, such as pressing a button.

9 Show All Persons

Show All Persons



```
GET /demo/api/persons HTTP/1.1
Host: http://localhost:8080
Accept: application/json

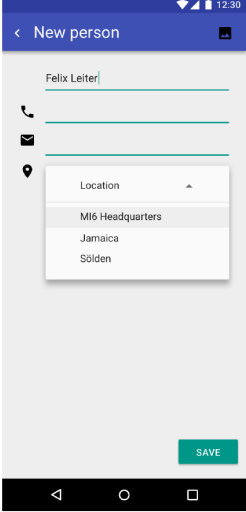
HTTP/1.1 200 OK
Date: Mon, 17 Aug 2020 12:30:08 GMT
Content-Type: application/json
Link: <.../demo/api/persons>; rel="createPerson"

[
  {
    "id": 1,
    "firstName": "James",
    "lastName": "Bond",
    "self": {
      "href": ".../demo/api/persons/1"
    }
  }
  ...
]
```

The client is currently in the `SShow all persons` state. The user interface changes to the corresponding screen, but no data is available yet. The client must call the URL it previously received with the relation name `"getAllPersons."` The client should know that this is a GET request. Consequently, he triggers the request to the server and receives the desired collection of already known persons in the response, which he can display accordingly. Each person includes a hyperlink as part of its representation, allowing the client to query a single person from the server in the next step. Let's assume the response message contains data on ten people; then the client will receive ten distinct hyperlinks for each person. This results in ten possible paths that the client can follow. For instance, in the user interface, it would enable the user to click on a person to view their details. This way, the user may see more data than in the view of all persons. Additionally, the response message includes the hyperlink with the relation name `"createPerson"` in the header area. This offers an eleventh option for the client. The client provides this action to the end user in the form of a button in the user interface. When the end user presses this button, he can create a new person.

10 Create a New Person

Create a New Person



```
POST /demo/api/persons HTTP/1.1
Host: http://localhost:8080
Content-Type: application/json

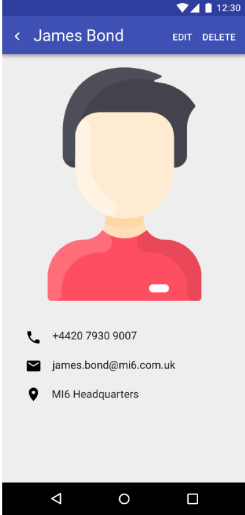
{
  "firstName": "Felix",
  "lastName": "Leiter",
}
```

```
HTTP/1.1 201 CREATED
Date: Mon, 17 Aug 2020 12:30:21 GMT
Location: .../demo/api/persons/3
```

Let's assume that the user wants to create a new person now. The application changes to the "Create a new person" state, and a form appears where the end user can enter the new person's data, such as first and last name, date of birth, and more. When the end user presses Save, the client triggers an HTTP request with the POST method, sending the new record in the content area. The server creates the new record in the database and includes a hyperlink in the response message to display the newly created record. As the client has only been provided this one follow-up action, it then switches to the "View a person" state.

11 Show a Single Person

Show a Single Person



James Bond EDIT DELETE

+4420 7930 9007

james.bond@mi6.com.uk

Mi6 Headquarters

```
GET /demo/api/persons/1 HTTP/1.1
Host: http://localhost:8080
Accept: application/json

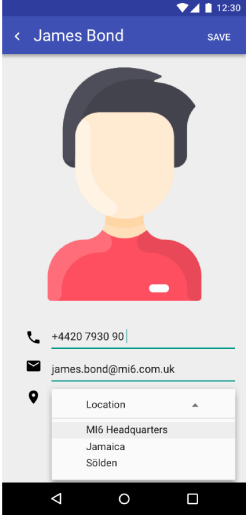
HTTP/1.1 200 OK
Date: Mon, 17 Aug 2020 12:30:28 GMT
Content-Type: application/json
Link: <.../demo/api/persons>; rel="getAllPersons"
Link: <.../demo/api/persons/1>; rel="updatePerson"
Link: <.../demo/api/persons/1>; rel="deletePerson"

{
  "id": 1,
  "firstName": "James",
  "lastName": "Bond",
  "self": {
    "href": ".../demo/api/persons/1"
  }
  ...
}
```

The client can reach the `SShow a person` state in two ways. Either the end-user has selected a person from the list of existing individuals, or a new person has just been created. There is now a URL for a specific person, and the client understands that it must use the HTTP verb GET. To display a single person, the client sends a GET request to the URL of the selected individual in the next step, receiving a response from the server that includes the resource representation as well as hyperlinks containing information about potential next steps. For instance, if the client is allowed to modify these resources, the server will inform the client via a hyperlink in the response header with the relation name `updatePerson`. In this case, the client could alert the end-user about the option to modify the data through an icon. The same applies to the capability of deleting a resource. If the server permits the deletion of this person, it sends a hyperlink with the relation name `deletePerson` in the response message, indicating to the client that an icon for deletion should also be displayed.

12 Update a Single Person

Update a Single Person



```
PUT /demo/api/persons/1 HTTP/1.1
Host: http://localhost:8080
Content-Type: application/json

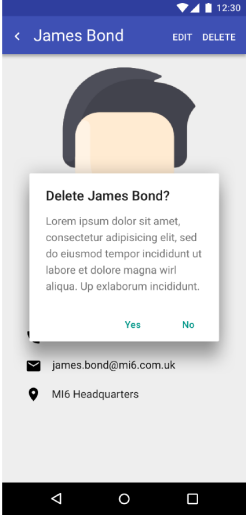
{
  "id": 1,
  "firstName": "James",
  "lastName": "Bond",
  ...
}
```

```
HTTP/1.1 204 NO CONTENT
Date: Mon, 17 Aug 2020 12:30:41 GMT
Link: </demo/api/persons/1>; rel="getPerson"
```

The Modify a Person state displays a form that shows the person's current data, which can be modified by the end-user. When the end-user presses Save, the client sends an HTTP message with the PUT verb to the server along with the modified data in the content area of the request message. The server then responds with only the hyperlink with the relation name "getSinglePerson." Consequently, the client must switch back to the SShow a person state next.

13 Delete a Single Person

Delete a Single Person



The screenshot shows a mobile application interface. At the top, a blue header bar contains a back arrow, the name 'James Bond', and 'EDIT' and 'DELETE' buttons. Below the header is a profile card for 'James Bond' with a placeholder image. A white dialog box with a grey border is centered over the profile card. The dialog has the title 'Delete James Bond?' and contains a paragraph of Lorem Ipsum text. At the bottom of the dialog are two buttons: 'Yes' (green) and 'No' (blue). Below the profile card, there is a contact information section showing an email address 'james.bond@mi6.com.uk' and a location 'MI6 Headquarters' with a location pin icon. The bottom of the screen shows the Android navigation bar.

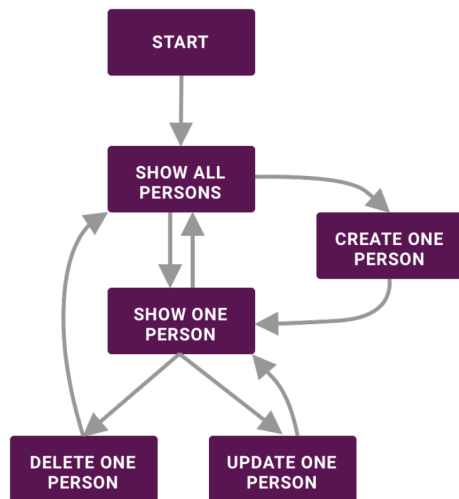
```
DELETE /demo/api/persons/1 HTTP/1.1
Host: http://localhost:8080
```

```
HTTP/1.1 204 NO CONTENT
Date: Mon, 17 Aug 2020 12:30:53 GMT
Link: <.../demo/api/persons>; rel="getAllPersons"
```

If the end user decides to delete the record, we arrive at the final state, "Delete a person." In this state, the end user may first see a dialog with a security question: "Do you really want to delete?" When this is confirmed, the client sends an HTTP request with the DELETE verb. The server no longer responds with a hyperlink to the person because they have been deleted; instead, it provides a hyperlink to "getAllPersons." Thus, the client transitions to the state SShow all persons."

14 The 5+1 States of a REST Application

The 5+1 States of a REST Application



Let's summarize once again and clarify the relationships with a graphic: Initially, our application is in the Start state, where the dispatcher is called. Next, the application transitions to show all available persons. Upon entering this state, it first requests the desired amount of resources from the server and then displays them on the screen. The application now waits for input from the user or for the end-user to decide which resources to view in more detail next. If the end-user selects a detailed view by choosing a specific resource, the application transitions to the "Detailed view of a resource." At the beginning of this state, the application retrieves the representation of the desired resources from the server. The end-user can switch to the "change resource" state by clicking a button if permitted. The interface changes to a dialog where the attributes of this resource can be modified. Once the end-user presses the Save button, a PUT request is triggered to the server, and the change is communicated to it. In the detailed view of a resource, the end-user can also press the button to delete the resource, provided the server has granted permission. The application then transitions to the state of SSecurity query before deleting the resource and presents the end-user with a dialog to decide whether to delete the resource. If the user opts to delete, a DELETE request is sent to the server. Afterward, the application transitions back to "Displaying the list of resources." As we can see, there is a corresponding application state for each URL in combination with an HTTP verb, which can also be found on the end-user interface. Transitions occur between states; that is, the application moves from one state to another, allowing for several alternatives in some cases. In other words, starting from one state, the application can end up in one of the subsequently offered states, depending on the end-user's decision. You can easily remember this with the example of the transition from the Detail View state to the two possible subsequent states: "edit the resource" and "delete the resource." Such a diagram

also illustrates that the application cannot jump between states arbitrarily. For example, there is no transition to change or delete a specific resource directly from the list of resources. The application must always go through the "Detailed view of a resource" state first. Furthermore, there is no transition to immediately create a new resource from the screen for editing a resource.

15 Formal Definition as Finite State Machine

Formal Definition as Finite State Machine

- $S = \{S_{Start}, S_{ShowAllPersons}, S_{CreateOnePerson}, S_{ShowOnePerson}, S_{DeleteOnePerson}, S_{UpdateOnePerson}\}$
- $\Sigma = \{E_{getAllPersons}, E_{createPerson}, E_{getPerson}, E_{updatePerson}, E_{deletePerson}\}$
- $S_0 = S_{Start}$
- $F = \emptyset$
- $\delta = \{(S_{Start}, E_{getAllPersons}, S_{ShowAllPersons}), (S_{ShowAllPersons}, E_{createPerson}, S_{CreateOnePerson}), (S_{ShowAllPersons}, E_{getPerson/self}, S_{ShowOnePerson}), (S_{ShowOnePerson}, E_{updatePerson}, S_{UpdateOnePerson}), (S_{ShowOnePerson}, E_{deletePerson}, S_{DeleteOnePerson}), \dots\}$

We can now redefine what we have just summarized in the picture using the language of finite state machines. The set of states includes Start, Show All Persons, Show One Person, Change One Person, and Delete One Person. The input symbols consist of the relation names from the hyperlinks, which represent actions on the user interface. These include getAllPersons, getPerson, createPerson, updatePerson, and deletePerson. You can see the allowed combinations for the relation Delta next to me.

16 Summary

Summary

- REST Systems can be modeled as Finite State Machine
- Only the client has a state – not the server!
- Think in states when you design a system

That's it for today. We approached REST as the architectural paradigm behind the World Wide Web from a theoretical standpoint, noting that a REST-based system can be seen as a finite automaton from a client's perspective. Next, we examined a practical example by considering the interaction between the client, responsible for the user interface, and the server, which provides the business logic and database functionality. When designing a REST-based application, it is beneficial to think about states and state transitions and discuss the entire system this way with other developers. It is essential to remember that the server developer needs to anticipate a client's possible states in advance. REST-based systems are defined by the server, limiting the client's ability to define a new state independently. A client's choices are restricted to selecting one of the available actions provided by the server and the end-user. Ultimately, it is up to the client how to present the resources to the end-user, meaning what the user interface will look like.

17 Literature

Literature

- Bill Burke: RESTful Java with JAX-RS 2.0 – Designing and Developing Distributed Web Services (Englisch). O'Reilly, 2013.
- Jim Webber, Savas Parastatidis, Ian Robinson: REST in Practise – Hypermedia and Systems Architecture. O'Reilly, 2010.